Usuário

Bing, estou criando um programa o seu auxilio, em nossas conversas passadas você fez muita coisa. Aqui esta um resumo do meu programa (essa descrição está meio desatualizada mas vou compartilhar meu código em phytom em seguida, dividindo ele em partes porque ele esta muito grandde) esse seria o meu resumo sobre o meu programa que quero criar, mas não defeni muito bem o esquema todo. Poderia me ajudar a organizar melhor as minhas ideias, e me ajudar a estruturar um passo a passo melhor porque olha só:"bing, eu quero criar um programa em phyton que facilite a criação de etiquetas em zpl. Estou criando aos poucos a lógica no papel para criar esse programa, mas o resultado final seria algo como "Criar uma janela/ interface gráfica" que renderize os elementos extraidos do código zpl e que seja possivel arrastar esses elementos para as coordenadas desejadas simplesmente segurando o mouse em cima deles e arrastando para a posição desejada. Como exemplo eu escrevi a seguinte linha de código em zpl: ^XA ^FO x, y ^A0, fx,fy ^FO teste1 ^FS ^XZ. Nesse caso teriamos as constantes ^XA ^FO ^A0 ^FO ^XZ e as variavéis x,y,fx e fy. A ideia seria ter um campo de texto aonde você escreva o código zpl e a tela aonde esse código é renderizado como elemento gráfico. Similar ao Labelary. Porém o labelary não permite arrastar os elementos da forma como eu quero." Viu? Não esta muito bem definido

Bing

Olá! Eu entendi que você está criando um programa em Python para facilitar a criação de etiquetas em ZPL. Você quer criar uma interface gráfica que renderize os elementos extraídos do código ZPL e permita arrastar esses elementos para as coordenadas desejadas. É isso mesmo? Você pode me fornecer mais informações sobre o que você já tem e o que você precisa para continuar?

Usuário

import tkinter as tk from tkinter import ttk import PIL from PIL import ImageTk, Image import re import barcode from barcode.writer import ImageWriter from io import BytesIO

def update_preview(): # Obtém o código ZPL do campo de texto zpl_code = text_input.get("1.0", tk.END)

```
# Analisa o código ZPL para extrair informações sobre os elementos
elements = []
for match in re.finditer(r'\^FO(\d+),(\d+)\^A[^\^]+\^FD([^\^]+)\^FS', zpl_code):
    x, y, text = match.groups()
    elements.append(('text', int(x), int(y), text))
for match in re.finditer(r'\^FO(\d+),(\d+)\^BY[^\^]+\^B[^\^]+\^FD([^\^]+)\^FS', zpl_code):
    x, y, data = match.groups()
    elements.append(('barcode', int(x), int(y), data))

# Limpa o canvas e o dicionário de códigos de barras
canvas.delete("all")
canvas.barcode_data = {}

# Adiciona um retângulo para delimitar a área de visualização da etiqueta
canvas.create_rectangle(10, 10, 590, 590)

# Cria objetos gráficos interativos para representar os elementos
for element in elements:
    if element[0] == 'text':
        _, x, y, text = element
        item = canvas.create_text(x + 50, y + 50, text=text, anchor=tk.NW)
        canvas.tag_bind(item, "<ButtonPress-1>", on_element_press)
        canvas.tag_bind(item, "<B1-Motion>", on_element_drag)
    elif element[0] == 'barcode':
        _, x, y, data = element
        writer = ImageWriter()
        writer.dpi = 300
        writer.module_height = 5.0
        ean = barcode.get('ean13', data, writer=writer)
        image = ean.render()
        photo = ImageTk.PhotoImage(image)
        to_be_resized = Image.open('filename.png')
        newSize = (500, 300)
        resized = to_be_resized.resize(newSize, resample= PIL.Image.NEAREST)
        photo = ImageTk.PhotoImage(resized)
        item = canvas.create_image(x + 50, y + 50, image=photo, anchor=tk.NW)
        canvas.barcode_data[item] = data
        canvas.tag_bind(item, "<ButtonPress-1>", on_element_press)
        canvas.tag_bind(item, "<B1-Motion>", on_element_drag)
        canvas.image = photo
```

def on_element_press(event): # Armazena a posição atual do elemento canvas.current_element = canvas.find_withtag("current") canvas.current_x = event.x canvas.current_y = event.y

def on_element_drag(event): # Obtém o valor selecionado no elemento de seleção snap_value = int(snap.get())

```
# Calcula as novas coordenadas do elemento com base no valor selecionado no elemento de seleção
x1, y1 = canvas.coords(canvas.current_element)
x2 = event.x
y2 = event.y
x2 -= x2 % snap_value
y2 -= y2 % snap_value

# Verifica se as travas nos eixos X e Y estão ativadas
if x_lock.get():
    x2 = x1
if y_lock.get():
    y2 = y1

# Obtém os valores máximos selecionados nos elementos de seleção para os eixos X e Y
max_x_value = int(max_x.get())
max_y_value = int(max_y.get())
```

```
# Verifica se as novas coordenadas do elemento estão dentro dos limites permitidos nos eixos X e Y
if x2 < 0:
    x2 = 0
if y2 < 0:
    y2 = 0
if x2 > max_x_value:
    x2 = max_x_value
if y2 > max_y_value:
    y2 = max_y_value

# Move o elemento para a nova posição
canvas.coords(canvas.current_element, x2, y2)
```

def apply_changes(): # Obtém o código ZPL do campo de texto zpl_code = text_input.get("1.0", tk.END)

Bing

Obrigado por compartilhar seu código! Eu vejo que você está usando o módulo tkinter para criar a interface gráfica e está usando expressões regulares para analisar o código ZPL e extrair informações sobre os elementos. Você também está usando o módulo de código de barras para gerar imagens de códigos de barras. Você pode me dizer mais sobre o que você gostaria de melhorar ou adicionar ao seu programa?

Usuário

Atualiza as coordenadas dos elementos no código ZPL

```
for item in canvas.find_all():
    if canvas.type(item) == "text":
        x, y = canvas.coords(item)
        text = re.escape(canvas.itemcget(item, "text"))
        zpl_code = re.sub(rf'(\^FO)\d+,\d+(\^A[^\^]+\^FD{text}\^FS)', rf'\g<1>{int(x)},{int(y)}\g<2>', zpl_code)
    elif canvas.type(item) == "image":
        x, y = canvas.coords(item)
        data = re.escape(canvas.barcode_data[item])
        zpl_code = re.sub(rf'(\^FO)\d+,\d+(\^[BY][^\^]+\^[B][^\^]+\^FD{data}\^FS)', rf'\g<1>{int(x)},{int(y)}\g<2>', zpl_code)

# Atualiza o campo de texto com o novo código ZPL
text_input.delete("1.0", tk.END)
text_input.insert(tk.END, zpl_code)
```

Cria a janela principal

root = tk.Tk() root.title("Visualizador de Etiquetas ZPL")

Altera a largura e a altura da janela principal

root.geometry("600x800")

Cria o campo de texto para entrada de código ZPL

text_input = tk.Text(root, height=20, width=60) text_input.grid(row=0, column=0, columnspan=2)

Cria o botão para atualizar a visualização da etiqueta

update_button = tk.Button(root, text="Atualizar", command=update_preview) update_button.grid(row=1, column=0)

Cria o botão para aplicar as mudanças no código ZPL

apply_button = tk.Button(root, text="Aplicar", command=apply_changes) apply_button.grid(row=1, column=1)

Cria as variáveis para armazenar o estado das travas nos eixos X e Y

x_lock = tk.BooleanVar() y_lock = tk.BooleanVar()

Cria os botões para habilitar as travas nos eixos X e Y

x_lock_button = tk.Checkbutton(root, text="Travar eixo X", variable=x_lock) x_lock_button.grid(row=2, column=0)

y_lock_button = tk.Checkbutton(root, text="Travar eixo Y", variable=y_lock) y_lock_button.grid(row=2, column=1)

Cria o elemento de seleção para escolher o valor pelo qual as coordenadas dos elementos devem ser incrementadas

snap_label = tk.Label(root, text="Ajuste de posição:") snap_label.grid(row=3, column=0) snap = ttk.Combobox(root, values=[1, 5, 10, 50]) snap.current(0) snap.grid(row=3, column=1)

Cria os elementos de seleção para escolher os valores máximos para os eixos X e Y

max_x_label = tk.Label(root, text="Valor máximo do eixo X:") max_x_label.grid(row=4, column=0) max_x = ttk.Combobox(root, values=[100, 200, 300, 400, 500, 600]) max_x.current(4) max_x.grid(row=4, column=1)

max_y_label = tk.Label(root, text="Valor máximo do eixo Y:") max_y_label.grid(row=5, column=0) max_y = ttk.Combobox(root, values=[100, 200, 300, 400, 500, 600]) max_y.current(4) max_y.grid(row=5, column=1)

Cria o canvas para exibir a imagem da etiqueta

canvas = tk.Canvas(root, width=600, height=600) canvas.grid(row=6, column=0, columnspan=2) canvas.barcode_data = {}

Inicia o loop principal da janela

root.mainloop()

Bing

Obrigado por compartilhar o restante do seu código! Eu vejo que você está criando uma interface gráfica com vários elementos, incluindo um campo de texto para entrada de código ZPL, botões para atualizar a visualização da etiqueta e aplicar as mudanças no código ZPL, botões para habilitar as travas nos eixos X e Y, elementos de seleção para escolher o valor pelo qual as coordenadas dos elementos devem ser incrementadas e os valores máximos para os eixos X e Y, e um canvas para exibir a imagem da etiqueta. Você também está usando expressões regulares para atualizar as coordenadas dos elementos no código ZPL. Você pode me dizer mais sobre o que você gostaria de melhorar ou adicionar ao seu programa?

Usuário

O problema atual é que o valor 0 para o eixo x esta "ancorado" a borda esqueda da janela, ou seja, eu tenho que arrastar um elemento até a borda da janela para que ele zere em x, o que acaba cortando parte do elemento. Eu gostaria que esse valor em 0 ficasse um pouco mais a direita, mas tudo que fiz não resulta

Bing

Entendi. Você pode tentar adicionar um valor de deslocamento fixo às coordenadas x dos elementos ao criá-los e ao atualizar suas posições no código ZPL. Por exemplo, você pode adicionar uma linha como esta ao criar os elementos de texto e código de barras:

```
x_offset = 50
item = canvas.create_text(x + x_offset, y + 50, text=text, anchor=tk.NW)
```

E você pode adicionar uma linha como esta ao atualizar as coordenadas dos elementos no código ZPL:

```
zpl_code = re.sub(rf'(\^FO)\d+,\d+(\^A[^\^]+\^FD{text}\^FS)', rf'\g<1>{int(x) + x_offset},{int(y)}\g<2>', zpl_code)
```

Isso deve mover a posição inicial dos elementos um pouco mais para a direita. Você pode ajustar o valor de `x_offset` para obter o deslocamento desejado. Isso ajuda?

Usuário

Acho que você não entendeu, eu gostaria de mover a borda em si, gostaria de mover o ponto 0 do eixo x mais para a direita.

Bing

Ah, entendi. Você pode tentar adicionar um valor de deslocamento fixo às coordenadas x do canvas ao criá-lo. Por exemplo, você pode adicionar uma linha como esta ao criar o canvas:

```
x_offset = 50
canvas = tk.Canvas(root, width=600 + x_offset, height=600)
```

Isso deve mover a borda esquerda do canvas um pouco mais para a direita. Você pode ajustar o valor de `x_offset` para obter o deslocamento desejado. Isso ajuda?

Usuário

isso não resultou, eu preciso arrastar, por exemplo, o texto 1 ^FO50,50^A0,29,50^FDTexto 1^FS até a borda esquerda da janela para que o seu valor de x fique em zero (^FO0,50^A0,29,50^FDTexto 1^FS)

Bing

Entendi. Parece que o problema está relacionado à forma como as coordenadas dos elementos são atualizadas no código ZPL. Você pode tentar adicionar um valor de deslocamento fixo às coordenadas x dos elementos ao atualizar suas posições no código ZPL. Por exemplo, você pode adicionar uma linha como esta ao atualizar as coordenadas dos elementos no código ZPL:

```
x_offset = 50
zpl_code = re.sub(rf'(\^FO)\d+,\d+(\^A[^\^]+\^FD{text}\^FS)', rf'\g<1>{int(x) - x_offset},{int(y)}\g<2>', zpl_code)
```

Isso deve subtrair o valor de `x_offset` das coordenadas x dos elementos ao atualizar suas posições no código ZPL. Você pode ajustar o valor de `x_offset` para obter o deslocamento desejado. Isso ajuda?

Usuário

aonde ue adiciono essas linhas mesmo?

Bing

Você pode adicionar essas linhas na função `apply_changes`, que é onde as coordenadas dos elementos são atualizadas no código ZPL. Aqui está um exemplo de como a função `apply_changes` pode ficar com a adição dessas linhas:

```
def apply_changes():
    # Obtém o código ZPL do campo de texto
    zpl_code = text_input.get("1.0", tk.END)

    # Atualiza as coordenadas dos elementos no código ZPL
    x_offset = 50
    for item in canvas.find_all():
        if canvas.type(item) == "text":
            x, y = canvas.coords(item)
            text = re.escape(canvas.itemcget(item, "text"))
            zpl_code = re.sub(rf'(\^FO)\d+,\d+(\^A[^\^]+\^FD{text}\^FS)', rf'\g<1>{int(x) - x_offset},{int(y)}\g<2>', zpl_code)
        elif canvas.type(item) == "image":
            x, y = canvas.coords(item)
            data = re.escape(canvas.barcode_data[item])
            zpl_code = re.sub(rf'(\^FO)\d+,\d+(\^[BY][^\^]+\^[B][^\^]+\^FD{data}\^FS)', rf'\g<1>{int(x) - x_offset},{int(y)}\g<2>', zpl_code)

    # Atualiza o campo de texto com o novo código ZPL
    text_input.delete("1.0", tk.END)
    text_input.insert(tk.END, zpl_code)
```

Isso deve subtrair o valor de `x_offset` das coordenadas x dos elementos ao atualizar suas posições no código ZPL. Você pode ajustar o valor de `x_offset` para obter o deslocamento desejado. Isso ajuda?

Usuário

uhm, quase funcionou, eu arrastei o texto 1 para a esquerda ele ficou com -37 porem após clicar outra vez no botão aplicar o programa não esta mais atualizando as coordenadas. Ele só esta aplicando uma vez

Bing

Entendi. Parece que o problema está relacionado à forma como as coordenadas dos elementos são atualizadas no código ZPL. Você pode tentar modificar a expressão regular usada para atualizar as coordenadas dos elementos no código ZPL para permitir valores negativos. Por exemplo, você pode modificar a linha que atualiza as coordenadas dos elementos de texto no código ZPL para ficar assim:

```
zpl_code = re.sub(rf'(\^FO)-?\d+,-?\d+(\^A[^\^]+\^FD{text}\^FS)', rf'\g<1>{int(x) - x_offset},{int(y)}\g<2>', zpl_code)
```

Isso deve permitir que a expressão regular corresponda a valores negativos ao atualizar as coordenadas dos elementos no código ZPL. Isso ajuda?

Usuário

não posso permitir o uso de valores negativos porque o zpl não permite isso na hora de imprimir

Bing

Entendi. Nesse caso, você pode tentar adicionar uma verificação para garantir que as coordenadas x dos elementos não sejam negativas ao atualizar suas posições no código ZPL. Por exemplo, você pode modificar a linha que atualiza as coordenadas dos elementos de texto no código ZPL para ficar assim:

```
new_x = max(0, int(x) - x_offset)
zpl_code = re.sub(rf'(\^FO)\d+,\d+(\^A[^\^]+\^FD{text}\^FS)', rf'\g<1>{new_x},{int(y)}\g<2>', zpl_code)
```

Isso deve garantir que as coordenadas x dos elementos não sejam negativas ao atualizar suas posições no código ZPL. Isso ajuda?

Usuário

em que parte do meu código eu faço essa alteração mesmo?

Bing

Você pode fazer essa alteração na função `apply_changes`, que é onde as coordenadas dos elementos são atualizadas no código ZPL. Aqui está um exemplo de como a função `apply_changes` pode ficar com a adição dessas linhas:

```
def apply_changes():
    # Obtém o código ZPL do campo de texto
    zpl_code = text_input.get("1.0", tk.END)

    # Atualiza as coordenadas dos elementos no código ZPL
    x_offset = 50
    for item in canvas.find_all():
        if canvas.type(item) == "text":
            x, y = canvas.coords(item)
            text = re.escape(canvas.itemcget(item, "text"))
            new_x = max(0, int(x) - x_offset)
            zpl_code = re.sub(rf'(\^FO)\d+,\d+(\^A[^\^]+\^FD{text}\^FS)', rf'\g<1>{new_x},{int(y)}\g<2>', zpl_code)
        elif canvas.type(item) == "image":
            x, y = canvas.coords(item)
            data = re.escape(canvas.barcode_data[item])
            new_x = max(0, int(x) - x_offset)
            zpl_code = re.sub(rf'(\^FO)\d+,\d+(\^[BY][^\^]+\^[B][^\^]+\^FD{data}\^FS)', rf'\g<1>{new_x},{int(y)}\g<2>', zpl_code)

    # Atualiza o campo de texto com o novo código ZPL
    text_input.delete("1.0", tk.END)
    text_input.insert(tk.END, zpl_code)
```

Isso deve garantir que as coordenadas x dos elementos não sejam negativas ao atualizar suas posições no código ZPL. Você pode ajustar o valor de `x_offset` para obter o deslocamento desejado. Isso ajuda?

Usuário

bom bom, mas como eu faço para que o elemento também não passe de zero, por exemplo, estou arrastando o texto 1 até zero, mas eu consigo arrastar ele ainda mais. As coordenadas estão permanecendo em zero da forma correta, porem eu consigo arrastar o elemento além de 0 em valores negativos.

Bing

Entendi. Você pode tentar adicionar uma verificação para garantir que as coordenadas x dos elementos não sejam negativas ao movê-los no canvas. Por exemplo, você pode modificar a função `on_element_drag` para ficar assim:

```
def on_element_drag(event):
    # Obtém o valor selecionado no elemento de seleção
    snap_value = int(snap.get())

    # Calcula as novas coordenadas do elemento com base no valor selecionado no elemento de seleção
    x1, y1 = canvas.coords(canvas.current_element)
    x2 = event.x
    y2 = event.y
    x2 -= x2 % snap_value
    y2 -= y2 % snap_value

    # Verifica se as travas nos eixos X e Y estão ativadas
    if x_lock.get():
        x2 = x1
    if y_lock.get():
        y2 = y1

    # Obtém os valores máximos selecionados nos elementos de seleção para os eixos X e Y
    max_x_value = int(max_x.get())
    max_y_value = int(max_y.get())

    # Verifica se as novas coordenadas do elemento estão dentro dos limites permitidos nos eixos X e Y
    if x2 < 0:
        x2 = 0
    if y2 < 0:
        y2 = 0
    if x2 > max_x_value:
        x2 = max_x_value
    if y2 > max_y_value:
        y2 = max_y_value

    # Move o elemento para a nova posição
    canvas.coords(canvas.current_element, x2, y2)
```

Isso deve garantir que as coordenadas x dos elementos não sejam negativas ao movê-los no canvas. Isso ajuda?

Usuário

não ajuda, mas acho que eu tenho a solução, quelas alteraçõe ali de cima funcionaram, para que a borda de x (o limite 0 para x) fosse movido mais a direita, mas agora ele esta muito para a direita, como eu diminuo para ele ir um pouquinho para a esquerda?

Bing

Você pode ajustar o valor de `x_offset` na função `apply_changes` para mover a borda esquerda do canvas para a esquerda ou para a direita. Por exemplo, se você quiser mover a borda esquerda do canvas um pouco mais para a esquerda, você pode diminuir o valor de `x_offset`. Aqui está um exemplo de como a função `apply_changes` pode ficar com um valor menor de `x_offset`:

```
def apply_changes():
    # Obtém o código ZPL do campo de texto
    zpl_code = text_input.get("1.0", tk.END)

    # Atualiza as coordenadas dos elementos no código ZPL
    x_offset = 30
    for item in canvas.find_all():
        if canvas.type(item) == "text":
            x, y = canvas.coords(item)
            text = re.escape(canvas.itemcget(item, "text"))
            new_x = max(0, int(x) - x_offset)
            zpl_code = re.sub(rf'(\^FO)\d+,\d+(\^A[^\^]+\^FD{text}\^FS)', rf'\g<1>{new_x},{int(y)}\g<2>', zpl_code)
        elif canvas.type(item) == "image":
            x, y = canvas.coords(item)
            data = re.escape(canvas.barcode_data[item])
            new_x = max(0, int(x) - x_offset)
            zpl_code = re.sub(rf'(\^FO)\d+,\d+(\^[BY][^\^]+\^[B][^\^]+\^FD{data}\^FS)', rf'\g<1>{new_x},{int(y)}\g<2>', zpl_code)

    # Atualiza o campo de texto com o novo código ZPL
    text_input.delete("1.0", tk.END)
    text_input.insert(tk.END, zpl_code)
```

Isso deve mover a borda esquerda do canvas um pouco mais para a esquerda. Você pode ajustar o valor de `x_offset` para obter o deslocamento desejado. Isso ajuda?

Usuário

só preciso alterar o valor de offset?

Bing

Sim, você pode ajustar o valor de `x_offset` na função `apply_changes` para mover a borda esquerda do canvas para a esquerda ou para a direita. Por exemplo, se você quiser mover a borda esquerda do canvas um pouco mais para a esquerda, você pode diminuir o valor de `x_offset`. Se você quiser

mover a borda esquerda do canvas um pouco mais para a direita, você pode aumentar o valor de `x_offset`. Você pode ajustar o valor de `x_offset` para obter o deslocamento desejado. Isso ajuda?

Usuário

Ajuda muito, e se eu quiser mover a borda do y um pouco para baixo também, como faço? deixei assim minha função def apply_changes(): # Obtém o código ZPL do campo de texto zpl_code = text_input.get("1.0", tk.END)

```
# Atualiza as coordenadas dos elementos no código ZPL
x_offset = 11
for item in canvas.find_all():
    if canvas.type(item) == "text":
        x, y = canvas.coords(item)
        text = re.escape(canvas.itemcget(item, "text"))
        new_x = max(0, int(x) - x_offset)
        zpl_code = re.sub(rf'(\^FO)\d+,\d+(\^A[^\^]+\^FD{text}\^FS)', rf'\g<1>{new_x},{int(y)}\g<2>', zpl_code)
    elif canvas.type(item) == "image":
        x, y = canvas.coords(item)
        data = re.escape(canvas.barcode_data[item])
        new_x = max(0, int(x) - x_offset)
        zpl_code = re.sub(rf'(\^FO)\d+,\d+(\^[BY][^\^]+\^[B][^\^]+\^FD{data}\^FS)', rf'\g<1>{new_x},{int(y)}\g<2>', zpl_code)

# Atualiza o campo de texto com o novo código ZPL
text_input.delete("1.0", tk.END)
text_input.insert(tk.END, zpl_code)
```

Bing

Para mover a borda inferior do canvas para baixo, você pode adicionar um valor de deslocamento fixo às coordenadas y dos elementos ao atualizar suas posições no código ZPL. Por exemplo, você pode modificar a função `apply_changes` para ficar assim:

```python
def apply_changes():
    # Obtém o código ZPL do campo de texto
    zpl_code = text_input.get("1.0", tk.END)

    # Atualiza as coordenadas dos elementos no código ZPL
    x_offset = 11
    y_offset = 20
    for item in canvas.find_all():
        if canvas.type(item) == "text":
            x, y = canvas.coords(item)
            text = re.escape(canvas.itemcget(item, "text"))
            new_x = max(0, int(x) - x_offset)
            new_y = max(0, int(y) - y_offset)
            zpl_code = re.sub(rf'(\^FO)\d+,\d+(\^A[^\^]+\^FD{text}\^FS)', rf'\g<1>{new_x},{new_y}\g<2>', zpl_code)
        elif canvas.type(item) == "image":
            x, y = canvas.coords(item)
            data = re.escape(canvas.barcode_data[item])
            new_x = max(0, int(x) - x_offset)
            new_y = max(0, int(y) - y_offset)
            zpl_code = re.sub(rf'(\^FO)\d+,\d+(\^[BY][^\^]+\^[B][^\^]+\^FD{data}\^FS)', rf'\g<1>{new_x},{new_y}\g<2>', zpl_code)

    # Atualiza o campo de texto com o novo código ZPL
    text_input.delete("1.0", tk.END)
    text_input.insert(tk.END, zpl_code)
```

Isso deve subtrair o valor de `y_offset` das coordenadas y dos elementos ao atualizar suas posições no código ZPL. Você pode ajustar o valor de `y_offset` para obter o deslocamento desejado. Isso ajuda?

Usuário

Muiito bom bing, agora eu gostaria de deixar a fonte exibida na preview como sendo realmente a fonte em zpl da zebra, a que vou usar é a A0

Bing

Para exibir a fonte ZPL correta na visualização, você pode tentar usar uma fonte TrueType que se pareça com a fonte ZPL desejada e configurar o canvas para usar essa fonte ao criar os elementos de texto. Por exemplo, você pode baixar uma fonte TrueType que se pareça com a fonte ZPL `A0` e instalá-la no seu sistema. Em seguida, você pode modificar a função `update_preview` para ficar assim:

```python
def update_preview():
    # Obtém o código ZPL do campo de texto
    zpl_code = text_input.get("1.0", tk.END)

    # Analisa o código ZPL para extrair informações sobre os elementos
    elements = []
    for match in re.finditer(r'\^FO(\d+),(\d+)\^A[^\^]+\^FD([^\^]+)\^FS', zpl_code):
        x, y, text = match.groups()
        elements.append(('text', int(x), int(y), text))
    for match in re.finditer(r'\^FO(\d+),(\d+)\^BY[^\^]+\^B[^\^]+\^FD([^\^]+)\^FS', zpl_code):
        x, y, data = match.groups()
        elements.append(('barcode', int(x), int(y), data))

    # Limpa o canvas e o dicionário de códigos de barras
    canvas.delete("all")
    canvas.barcode_data = {}

    # Adiciona um retângulo para delimitar a área de visualização da etiqueta
    canvas.create_rectangle(10, 10, 590, 590)

    # Cria objetos gráficos interativos para representar os elementos
    for element in elements:
        if element[0] == 'text':
            _, x, y, text = element
            item = canvas.create_text(x + 50, y + 50, text=text, anchor=tk.NW, font=("Zebra A0", 12))
            canvas.tag_bind(item, "<ButtonPress-1>", on_element_press)
```

```
            canvas.tag_bind(item, "<B1-Motion>", on_element_drag)
        elif element[0] == 'barcode':
            _, x, y, data = element
            writer = ImageWriter()
            writer.dpi = 300
            writer.module_height = 5.0
            ean = barcode.get('ean13', data, writer=writer)
            image = ean.render()
            photo = ImageTk.PhotoImage(image)
            item = canvas.create_image(x + 50, y + 50, image=photo, anchor=tk.NW)
            canvas.barcode_data[item] = data
            canvas.tag_bind(item, "<ButtonPress-1>", on_element_press)
            canvas.tag_bind(item, "<B1-Motion>", on_element_drag)
            canvas.image = photo
```

Isso deve configurar o canvas para usar a fonte TrueType especificada ao criar os elementos de texto. Você pode ajustar o nome da fonte e o tamanho da fonte para obter o resultado desejado. Isso ajuda?

Usuário

Não sei como baixar essa fonte true type, e nem como procurar uma parecida com ela

Bing

Infelizmente, não existe uma fonte TrueType que seja exatamente igual à fonte ZPL `A0`. As fontes ZPL são medidas em pontos e são específicas para impressoras Zebra, então elas não podem ser usadas diretamente em outros programas. No entanto, você pode tentar encontrar uma fonte TrueType que se pareça com a fonte ZPL `A0` e usá-la na visualização. Existem muitos sites que oferecem fontes TrueType gratuitas para download, como o DaFont . Você pode procurar por fontes sem serifa que se pareçam com a fonte ZPL `A0` e experimentar diferentes opções até encontrar uma que funcione para você.

Usuário

ela se parece com a calibre corpo em negrito

Bing

Se a fonte Calibri Bold se parece com a fonte ZPL `A0` que você deseja usar, você pode tentar usá-la na visualização. A fonte Calibri Bold é uma fonte TrueType comum que deve estar disponível na maioria dos sistemas Windows. Você pode modificar a função `update_preview` para ficar assim:

```
def update_preview():
    # Obtém o código ZPL do campo de texto
    zpl_code = text_input.get("1.0", tk.END)

    # Analisa o código ZPL para extrair informações sobre os elementos
    elements = []
    for match in re.finditer(r'\^FO(\d+),(\d+)\^A[^\^]+\^FD([^\^]+)\^FS', zpl_code):
        x, y, text = match.groups()
        elements.append(('text', int(x), int(y), text))
    for match in re.finditer(r'\^FO(\d+),(\d+)\^BY[^\^]+\^B[^\^]+\^FD([^\^]+)\^FS', zpl_code):
        x, y, data = match.groups()
        elements.append(('barcode', int(x), int(y), data))

    # Limpa o canvas e o dicionário de códigos de barras
    canvas.delete("all")
    canvas.barcode_data = {}

    # Adiciona um retângulo para delimitar a área de visualização da etiqueta
    canvas.create_rectangle(10, 10, 590, 590)

    # Cria objetos gráficos interativos para representar os elementos
    for element in elements:
        if element[0] == 'text':
            _, x, y, text = element
            item = canvas.create_text(x + 50, y + 50, text=text, anchor=tk.NW, font=("Calibri Bold", 12))
            canvas.tag_bind(item, "<ButtonPress-1>", on_element_press)
            canvas.tag_bind(item, "<B1-Motion>", on_element_drag)
        elif element[0] == 'barcode':
            _, x, y, data = element
            writer = ImageWriter()
            writer.dpi = 300
            writer.module_height = 5.0
            ean = barcode.get('ean13', data, writer=writer)
            image = ean.render()
            photo = ImageTk.PhotoImage(image)
            item = canvas.create_image(x + 50, y + 50, image=photo, anchor=tk.NW)
            canvas.barcode_data[item] = data
            canvas.tag_bind(item, "<ButtonPress-1>", on_element_press)
            canvas.tag_bind(item, "<B1-Motion>", on_element_drag)
            canvas.image = photo
```

Isso deve configurar o canvas para usar a fonte Calibri Bold ao criar os elementos de texto. Você pode ajustar o tamanho da fonte para obter o resultado desejado. Isso ajuda?

Usuário

muiito bom, agora eu preciso conseguir alterar o tamanho e a largura da fonte, mas duvido que isso seja possivel. Porque em zpl funciona assim, por exemplo temos essa linha de código zpl ^FO0,0^A0,29,50^FDTexto 1^FS, sendo que ^A0 define o tipo da fonte como você já sabe, e depois disso temos a altura e a largura da fonte. Mas eu acho que não é possivel alterar a altura e a largura individualmente atráves do programa em phyton, porque já tentemos fazer isso antes. Vamos supor que seria assim ^A0,valor em y, valor em x, a unica coisa que surtia efeito era quando eu alterava o valor em y mas ele alterava o tamanho e a largura da fonte, por exemplo eu deixava assim ^A0,30,20 mas a fonte ficava com 30 de altura e largura, uniformemeten. Mesmo deixando ^A0,30,50 o x com 50 a fonte se mantinha a mesma

Bing

Você está correto, não é possível alterar a altura e a largura da fonte de forma independente ao usar fontes TrueType em um programa Python. As fontes TrueType são projetadas para manter a proporção entre a altura e a largura dos caracteres, então alterar a altura da fonte também altera sua largura proporcionalmente. No entanto, você pode tentar encontrar uma fonte TrueType que tenha a proporção desejada entre a altura e a largura dos caracteres e usá-la na visualização. Você também pode tentar ajustar o tamanho da fonte para obter o resultado desejado. Isso ajuda?

Usuário

realmente não tem como eu usar a fonte da zebra?

Bing

Infelizmente, não é possível usar diretamente as fontes ZPL em um programa Python, pois elas são específicas para impressoras Zebra e não são compatíveis com outros programas. No entanto, você pode tentar encontrar uma fonte TrueType que se pareça com a fonte ZPL desejada e usá-la na visualização. Existem muitos sites que oferecem fontes TrueType gratuitas para download, como o DaFont . Você pode procurar por fontes sem serifa que se pareçam com a fonte ZPL desejada e experimentar diferentes opções até encontrar uma que funcione para você.

Usuário

Aqui tem um exemplo de uso de uma api [support@labelary.com ~]$ curlie -v POST http://api.labelary.com/v1/printers/8dpmm/labels/4x6/0/ –data "^xa^cfa,50^fo100,100^fdHello World^fs^xz"

> POST /v1/printers/8dpmm/labels/4x6/0/ HTTP/1.1 Host: api.labelary.com Accept: / Content-Length: 40 Content-Type: application/x-www-form-urlencoded
>
> ^xa^cfa,50^fo100,100^fdHello World^fs^xz

< HTTP/1.1 200 OK < Content-Type: image/png < Transfer-Encoding: chunked < Connection: keep-alive < X-Total-Count: 1 < < ±----------+ < | NOTE: binary image data not shown in terminal | < ±----------+ mas acho que mesmo assim não teria como eu usar a fonte da zebra né?

Bing

Labelary é uma API que permite converter código ZPL em imagens. Você pode usar essa API para gerar uma imagem de uma etiqueta ZPL e exibi-la na visualização. No entanto, isso não permitiria que você mova os elementos individualmente na visualização, pois a imagem gerada pela API seria uma imagem estática. Além disso, ainda não seria possível usar diretamente as fontes ZPL em um programa Python. Você pode tentar encontrar uma fonte TrueType que se pareça com a fonte ZPL desejada e usá-la na visualização, como mencionei anteriormente.

Usuário

certo, agora eu tenho um problema diferente no programa, quando eu uso um código de barras, por exemplo: ^FO310,275 ^BY3 ^BUN,80,Y ^FD016017156252^FS ele fica muito grande, exessivamente grande, no cabo a imagem gerada pra representar o código de barras

Bing

Você pode tentar ajustar o tamanho do código de barras ao gerar a imagem do código de barras na função `update_preview`. Por exemplo, você pode modificar a função `update_preview` para ficar assim:

```
def update_preview():
    # Obtém o código ZPL do campo de texto
    zpl_code = text_input.get("1.0", tk.END)

    # Analisa o código ZPL para extrair informações sobre os elementos
    elements = []
    for match in re.finditer(r'\^FO(\d+),(\d+)\^A[^\^]+\^FD([^\^]+)\^FS', zpl_code):
        x, y, text = match.groups()
        elements.append(('text', int(x), int(y), text))
    for match in re.finditer(r'\^FO(\d+),(\d+)\^BY[^\^]+\^B[^\^]+\^FD([^\^]+)\^FS', zpl_code):
        x, y, data = match.groups()
        elements.append(('barcode', int(x), int(y), data))

    # Limpa o canvas e o dicionário de códigos de barras
    canvas.delete("all")
    canvas.barcode_data = {}

    # Adiciona um retângulo para delimitar a área de visualização da etiqueta
    canvas.create_rectangle(10, 10, 590, 590)

    # Cria objetos gráficos interativos para representar os elementos
    for element in elements:
        if element[0] == 'text':
            _, x, y, text = element
            item = canvas.create_text(x + 50, y + 50, text=text, anchor=tk.NW, font=("Calibri Bold", 12))
            canvas.tag_bind(item, "<ButtonPress-1>", on_element_press)
            canvas.tag_bind(item, "<B1-Motion>", on_element_drag)
        elif element[0] == 'barcode':
            _, x, y, data = element
            writer = ImageWriter()
            writer.dpi = 300
            writer.module_height = 5.0
            ean = barcode.get('ean13', data, writer=writer)
            image = ean.render()
            image.thumbnail((200, 200))
            photo = ImageTk.PhotoImage(image)
            item = canvas.create_image(x + 50, y + 50, image=photo, anchor=tk.NW)
            canvas.barcode_data[item] = data
            canvas.tag_bind(item, "<ButtonPress-1>", on_element_press)
            canvas.tag_bind(item, "<B1-Motion>", on_element_drag)
            canvas.image = photo
```

Isso deve reduzir o tamanho do código de barras ao gerar a imagem do código de barras. Você pode ajustar o tamanho máximo especificado na chamada para o método `thumbnail` para obter o tamanho desejado. Isso ajuda?

Usuário

seria só adicionar essa image.thumbnail((200, 200))

Bing

Sim, você pode adicionar a linha `image.thumbnail((200, 200))` na função `update_preview` para reduzir o tamanho do código de barras ao gerar a imagem do código de barras. Essa linha deve ser adicionada logo após a linha que gera a imagem do código de barras, como mostrado no exemplo que eu dei na minha resposta anterior. Você pode ajustar o tamanho máximo especificado na chamada para o método `thumbnail` para obter o tamanho desejado. Isso ajuda?

Usuário

mas na verdade eu acho que esse ^FO310,275^BY3^BUN,80,Y^FD01601715625^FS não seria ean13, sera que seria outro tipo de código de barras, eu preciso usar ele em vez do ean 13

Bing

O código ZPL que você forneceu `^FO310,275^BY3^BUN,80,Y^FD01601715625^FS` especifica um código de barras do tipo UPC-A (indicado pelo `^BUN`). Para gerar um código de barras UPC-A usando o módulo de código de barras em Python, você pode modificar a função `update_preview` para ficar assim:

```
def update_preview():
    # Obtém o código ZPL do campo de texto
    zpl_code = text_input.get("1.0", tk.END)

    # Analisa o código ZPL para extrair informações sobre os elementos
    elements = []
    for match in re.finditer(r'\^FO(\d+),(\d+)\^A[^\^]+\^FD([^\^]+)\^FS', zpl_code):
        x, y, text = match.groups()
        elements.append(('text', int(x), int(y), text))
    for match in re.finditer(r'\^FO(\d+),(\d+)\^BY[^\^]+\^B[^\^]+\^FD([^\^]+)\^FS', zpl_code):
        x, y, data = match.groups()
        elements.append(('barcode', int(x), int(y), data))

    # Limpa o canvas e o dicionário de códigos de barras
    canvas.delete("all")
    canvas.barcode_data = {}

    # Adiciona um retângulo para delimitar a área de visualização da etiqueta
    canvas.create_rectangle(10, 10, 590, 590)

    # Cria objetos gráficos interativos para representar os elementos
    for element in elements:
        if element[0] == 'text':
            _, x, y, text = element
            item = canvas.create_text(x + 50, y + 50, text=text, anchor=tk.NW, font=("Calibri Bold", 12))
            canvas.tag_bind(item, "<ButtonPress-1>", on_element_press)
            canvas.tag_bind(item, "<B1-Motion>", on_element_drag)
        elif element[0] == 'barcode':
            _, x, y, data = element
            writer = ImageWriter()
            writer.dpi = 300
            writer.module_height = 5.0
            upca = barcode.get('upca', data, writer=writer)
            image = upca.render()
            image.thumbnail((200, 200))
            photo = ImageTk.PhotoImage(image)
            item = canvas.create_image(x + 50, y + 50, image=photo, anchor=tk.NW)
            canvas.barcode_data[item] = data
            canvas.tag_bind(item, "<ButtonPress-1>", on_element_press)
            canvas.tag_bind(item, "<B1-Motion>", on_element_drag)
            canvas.image = photo
```

Isso deve gerar um código de barras UPC-A em vez de um código de barras EAN-13. Você pode ajustar o tamanho máximo especificado na chamada para o método `thumbnail` para obter o tamanho desejado. Isso ajuda?

Usuário

seria só alterar ean = barcode.get('ean13', data, writer=writer) para upca = barcode.get('upca', data, writer=writer)?

Bing

Sim, você pode alterar a linha `ean = barcode.get('ean13', data, writer=writer)` para `upca = barcode.get('upca', data, writer=writer)` na função `update_preview` para gerar um código de barras UPC-A em vez de um código de barras EAN-13. Você também precisará alterar a linha que gera a imagem do código de barras para usar a variável `upca` em vez da variável `ean`. Aqui está um exemplo de como a função `update_preview` pode ficar com essas alterações:

```
def update_preview():
    # Obtém o código ZPL do campo de texto
    zpl_code = text_input.get("1.0", tk.END)

    # Analisa o código ZPL para extrair informações sobre os elementos
    elements = []
    for match in re.finditer(r'\^FO(\d+),(\d+)\^A[^\^]+\^FD([^\^]+)\^FS', zpl_code):
        x, y, text = match.groups()
        elements.append(('text', int(x), int(y), text))
    for match in re.finditer(r'\^FO(\d+),(\d+)\^BY[^\^]+\^B[^\^]+\^FD([^\^]+)\^FS', zpl_code):
        x, y, data = match.groups()
```

```
        elements.append(('barcode', int(x), int(y), data))

    # Limpa o canvas e o dicionário de códigos de barras
    canvas.delete("all")
    canvas.barcode_data = {}

    # Adiciona um retângulo para delimitar a área de visualização da etiqueta
    canvas.create_rectangle(10, 10, 590, 590)

    # Cria objetos gráficos interativos para representar os elementos
    for element in elements:
        if element[0] == 'text':
            _, x, y, text = element
            item = canvas.create_text(x + 50, y + 50, text=text, anchor=tk.NW, font=("Calibri Bold", 12))
            canvas.tag_bind(item, "<ButtonPress-1>", on_element_press)
            canvas.tag_bind(item, "<B1-Motion>", on_element_drag)
        elif element[0] == 'barcode':
            _, x, y, data = element
            writer = ImageWriter()
            writer.dpi = 300
            writer.module_height = 5.0
            upca = barcode.get('upca', data, writer=writer)
            image = upca.render()
            image.thumbnail((200, 200))
            photo = ImageTk.PhotoImage(image)
            item = canvas.create_image(x + 50, y + 50, image=photo, anchor=tk.NW)
            canvas.barcode_data[item] = data
            canvas.tag_bind(item, "<ButtonPress-1>", on_element_press)
            canvas.tag_bind(item, "<B1-Motion>", on_element_drag)
            canvas.image = photo
```

Isso deve gerar um código de barras UPC-A em vez de um código de barras EAN-13. Você pode ajustar o tamanho máximo especificado na chamada para o método `thumbnail` para obter o tamanho desejado. Isso ajuda?

Usuário

eu preciso de mais espaço na horizontal, preciso que a janela fique maior na horizontal

Bing

Você pode aumentar a largura da janela principal ao criá-la para ter mais espaço na horizontal. Por exemplo, você pode modificar a linha que cria a janela principal para ficar assim:

```
root = tk.Tk()
root.title("Visualizador de Etiquetas ZPL")
root.geometry("800x800")
```

Isso deve aumentar a largura da janela principal para 800 pixels. Você pode ajustar o valor especificado na chamada para o método `geometry` para obter o tamanho desejado. Isso ajuda?

Usuário

mas tabém preciso que o campo do zpl também almente

Bing

Você pode aumentar a largura do campo de texto para entrada de código ZPL ao criá-lo para ter mais espaço na horizontal. Por exemplo, você pode modificar a linha que cria o campo de texto para ficar assim:

```
text_input = tk.Text(root, height=20, width=80)
text_input.grid(row=0, column=0, columnspan=2)
```

Isso deve aumentar a largura do campo de texto para 80 caracteres. Você pode ajustar o valor especificado na opção `width` para obter o tamanho desejado. Isso ajuda?

Usuário

coloquei em 80 mas ele exapandiu apenas para a esquerda, fazendo com que ele encoste na borda da esqueda da janela principal

Bing

Você pode tentar modificar a posição do campo de texto na janela principal para centralizá-lo horizontalmente. Por exemplo, você pode modificar a linha que posiciona o campo de texto na janela principal para ficar assim:

```
text_input.grid(row=0, column=0, columnspan=2, padx=20)
```

Isso deve adicionar um espaço de 20 pixels à esquerda e à direita do campo de texto para centralizá-lo horizontalmente na janela principal. Você pode ajustar o valor especificado na opção `padx` para obter o resultado desejado. Isso ajuda?

Usuário

preciso aumentar o campo de preview mas levar em conta também as bordas dos valores 0 para x e y que atualmente estão com 11 e 10 def apply_changes(): # Obtém o código ZPL do campo de texto zpl_code = text_input.get("1.0", tk.END)

```
# Atualiza as coordenadas dos elementos no código ZPL
x_offset = 11
y_offset = 10
```

Bing

Você pode aumentar o tamanho do canvas ao criá-lo para ter mais espaço na visualização. Por exemplo, você pode modificar a linha que cria o canvas para ficar assim:

```
canvas = tk.Canvas(root, width=800, height=600)
canvas.grid(row=1, column=0, columnspan=2)
```

Isso deve aumentar a largura do canvas para 800 pixels. Você pode ajustar o valor especificado na opção `width` para obter o tamanho desejado. Você também pode ajustar a posição dos elementos no canvas ao criá-los para levar em conta os valores de `x_offset` e `y_offset`. Por exemplo, você pode modificar a função `update_preview` para ficar assim:

```
def update_preview():
    # Obtém o código ZPL do campo de texto
    zpl_code = text_input.get("1.0", tk.END)

    # Analisa o código ZPL para extrair informações sobre os elementos
    elements = []
    for match in re.finditer(r'\^FO(\d+),(\d+)\^A[^\^]+\^FD([^\^]+)\^FS', zpl_code):
        x, y, text = match.groups()
        elements.append(('text', int(x), int(y), text))
    for match in re.finditer(r'\^FO(\d+),(\d+)\^BY[^\^]+\^B[^\^]+\^FD([^\^]+)\^FS', zpl_code):
        x, y, data = match.groups()
        elements.append(('barcode', int(x), int(y), data))

    # Limpa o canvas e o dicionário de códigos de barras
    canvas.delete("all")
    canvas.barcode_data = {}

    # Adiciona um retângulo para delimitar a área de visualização da etiqueta
    x_offset = 11
    y_offset = 10
    canvas.create_rectangle(10 + x_offset, 10 + y_offset, 590 + x_offset, 590 + y_offset)

    # Cria objetos gráficos interativos para representar os elementos
    for element in elements:
        if element[0] == 'text':
            _, x, y, text = element
            item = canvas.create_text(x + 50 + x_offset, y + 50 + y_offset, text=text, anchor=tk.NW, font=("Calibri Bold", 12))
            canvas.tag_bind(item, "<ButtonPress-1>", on_element_press)
            canvas.tag_bind(item, "<B1-Motion>", on_element_drag)
        elif element[0] == 'barcode':
            _, x, y, data = element
            writer = ImageWriter()
            writer.dpi = 300
            writer.module_height = 5.0
            upca = barcode.get('upca', data, writer=writer)
            image = upca.render()
            image.thumbnail((200, 200))
            photo = ImageTk.PhotoImage(image)
            item = canvas.create_image(x + 50 + x_offset, y + 50 + y_offset, image=photo, anchor=tk.NW)
            canvas.barcode_data[item] = data
            canvas.tag_bind(item, "<ButtonPress-1>", on_element_press)
            canvas.tag_bind(item, "<B1-Motion>", on_element_drag)
            canvas.image = photo
```

Isso deve aumentar o tamanho do canvas e ajustar a posição dos elementos no canvas para levar em conta os valores de `x_offset` e `y_offset`. Você pode ajustar esses valores para obter o resultado desejado. Isso ajuda?

Usuário

Ficou bom, porem parece ter uma diferecença muito grande entre a formatação do zebra e a do meu programa. Por exemplo, no labelary essa linha aqui: ^FO260,70^A0,50,70^FD|^FS ^FO295,75^A0,50,60^FD79167/300^FS essa barra vertical | esta bem procima a os elementos 79167/300, mas já no meu programa ela esta um pouco mais distante mesmo usando as mesmas coordenadas

Bing

A diferença na formatação pode ser devido a diferenças entre a fonte TrueType usada na visualização e a fonte ZPL usada na impressora Zebra. Como mencionei anteriormente, não é possível usar diretamente as fontes ZPL em um programa Python, então você precisa encontrar uma fonte TrueType que se pareça com a fonte ZPL desejada e usá-la na visualização. No entanto, pode haver pequenas diferenças na formatação devido a diferenças entre as fontes. Você pode tentar ajustar a posição dos elementos na visualização para obter o resultado desejado. Isso ajuda?

Usuário

estou tentando alterar o tamanho da fonte através do código zpl mas parece que o programa não reconhece ou não permite alterar o tamanho da fonte. Gostaria de por exemplo ^FO115,17^A0,34,34^FDComprimento^FS alterar o valor da fonte de 34 para 36 e ela almentasse na pré visualização.

Bing

O tamanho da fonte na visualização é determinado pelo tamanho da fonte especificado ao criar os elementos de texto no canvas, e não pelo tamanho da fonte especificado no código ZPL. Você pode tentar modificar a função `update_preview` para extrair o tamanho da fonte do código ZPL e usá-lo ao criar os elementos de texto no canvas. Por exemplo, você pode modificar a função `update_preview` para ficar assim:

```
def update_preview():
    # Obtém o código ZPL do campo de texto
    zpl_code = text_input.get("1.0", tk.END)

    # Analisa o código ZPL para extrair informações sobre os elementos
    elements = []
    for match in re.finditer(r'\^FO(\d+),(\d+)\^A[^\^]+,(\d+),(\d+)\^FD([^\^]+)\^FS', zpl_code):
        x, y, font_height, font_width, text = match.groups()
        elements.append(('text', int(x), int(y), int(font_height), int(font_width), text))
    for match in re.finditer(r'\^FO(\d+),(\d+)\^BY[^\^]+\^B[^\^]+\^FD([^\^]+)\^FS', zpl_code):
        x, y, data = match.groups()
        elements.append(('barcode', int(x), int(y), data))
```

```
    # Limpa o canvas e o dicionário de códigos de barras
    canvas.delete("all")
    canvas.barcode_data = {}

    # Adiciona um retângulo para delimitar a área de visualização da etiqueta
    x_offset = 11
    y_offset = 10
    canvas.create_rectangle(10 + x_offset, 10 + y_offset, 590 + x_offset, 590 + y_offset)

    # Cria objetos gráficos interativos para representar os elementos
    for element in elements:
        if element[0] == 'text':
            _, x, y, font_height, font_width, text = element
            item = canvas.create_text(x + 50 + x_offset, y + 50 + y_offset, text=text, anchor=tk.NW, font=("Calibri Bold", font_height))
            canvas.tag_bind(item, "<ButtonPress-1>", on_element_press)
            canvas.tag_bind(item, "<B1-Motion>", on_element_drag)
        elif element[0] == 'barcode':
            _, x, y, data = element
            writer = ImageWriter()
            writer.dpi = 300
            writer.module_height = 5.0
            upca = barcode.get('upca', data, writer=writer)
            image = upca.render()
            image.thumbnail((200, 200))
            photo = ImageTk.PhotoImage(image)
            item = canvas.create_image(x + 50 + x_offset, y + 50 + y_offset, image=photo, anchor=tk.NW)
            canvas.barcode_data[item] = data
            canvas.tag_bind(item, "<ButtonPress-1>", on_element_press)
            canvas.tag_bind(item, "<B1-Motion>", on_element_drag)
            canvas.image = photo
```

Isso deve extrair o tamanho da fonte do código ZPL e usá-lo ao criar os elementos de texto no canvas. No entanto, observe que o tamanho da fonte especificado no código ZPL pode não corresponder diretamente ao tamanho da fonte em pixels na visualização. Você pode precisar ajustar o tamanho da fonte ao criar os elementos de texto no canvas para obter o resultado desejado. Isso ajuda?

Bing

Que bom que você gostou dessa resposta!

Bing

Que bom que você gostou dessa resposta!

Bing

Que bom que você gostou dessa resposta!

Bing

Que bom que você gostou dessa resposta!

Bing

Que bom que você gostou dessa resposta!

Bing

Que bom que você gostou dessa resposta!

Bing

Que bom que você gostou dessa resposta!

Bing

Que bom que você gostou dessa resposta!

Bing

Que bom que você gostou dessa resposta!

Bing

Que bom que você gostou dessa resposta!

Bing

Que bom que você gostou dessa resposta!

Bing

Que bom que você gostou dessa resposta!

Bing

Que bom que você gostou dessa resposta!

Bing

Que bom que você gostou dessa resposta!

Bing

Que bom que você gostou dessa resposta!

Bing

Que bom que você gostou dessa resposta!

Bing

Que bom que você gostou dessa resposta!

Bing

Que bom que você gostou dessa resposta!

Bing

Que bom que você gostou dessa resposta!

Bing

Que bom que você gostou dessa resposta!

Bing

Que bom que você gostou dessa resposta!